

Universidad de Santiago de Chile

Departamento de Ingeniería Informática

Fundamentos Aprendizaje Profundo Elaboración Modelos con MLP

Dr. Gonzalo Acuña L.

Marzo 2019



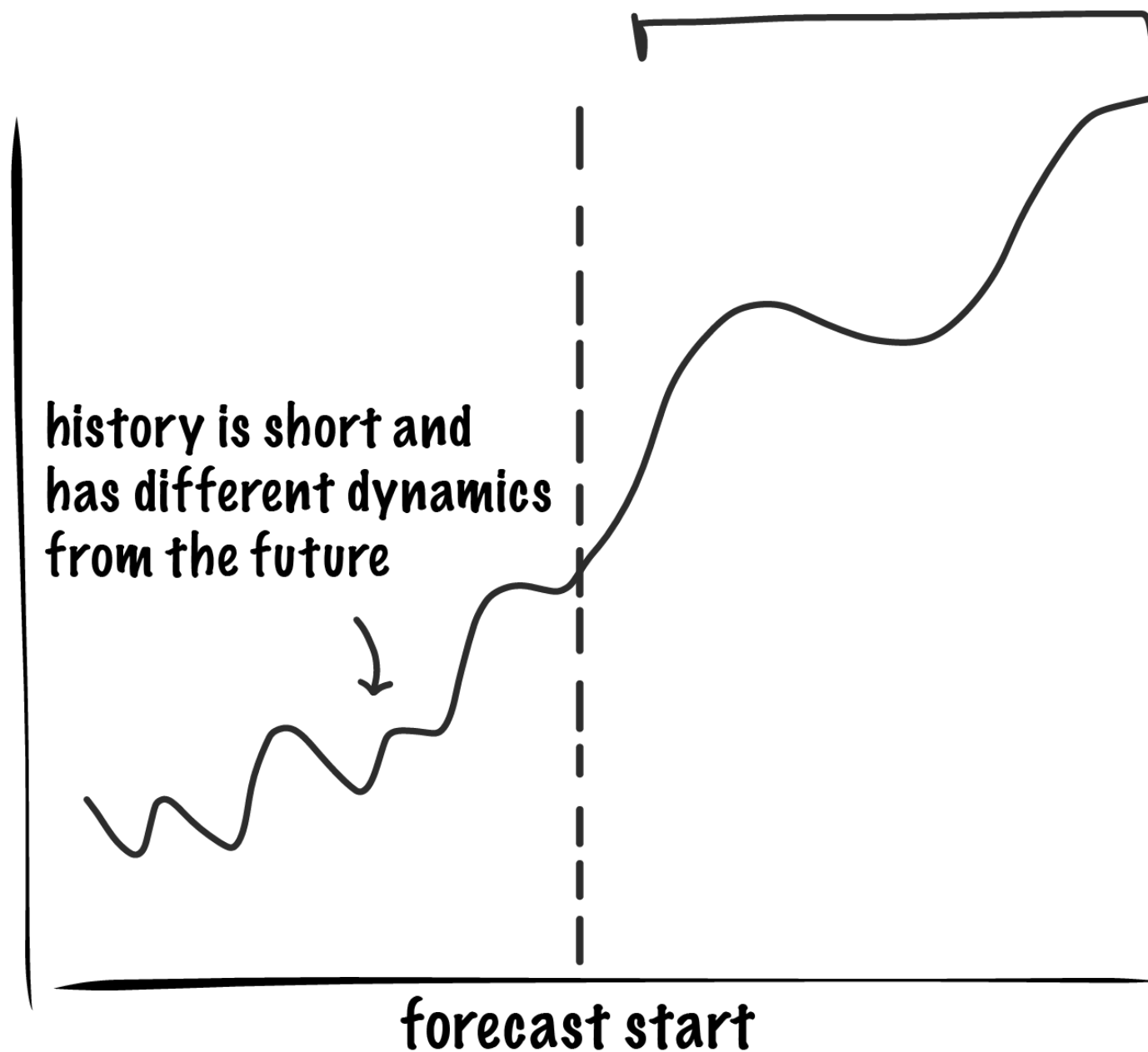
Pasos para elaboración de modelos con Perceptrones Multicapa

Paso 1: Exhaustivo análisis de sistema

- En este análisis se debe dejar establecido el número y tipo de variables de entrada y salida del modelo, la posibilidad de reducir la dimensión del problema disminuyendo el número de variables involucradas, etc...
- Es realmente imprescindible usar un modelo neuronal? Porqué no utilizar modelos clásicos existentes? **RED NEURONAL: SEGUNDA MEJOR SOLUCIÓN !!**
- Si se decide utilizar un modelo neuronal, se cuenta con los datos que representen adecuadamente el fenómeno a modelar y en la cantidad suficiente?

Paso 2: Preprocesamiento

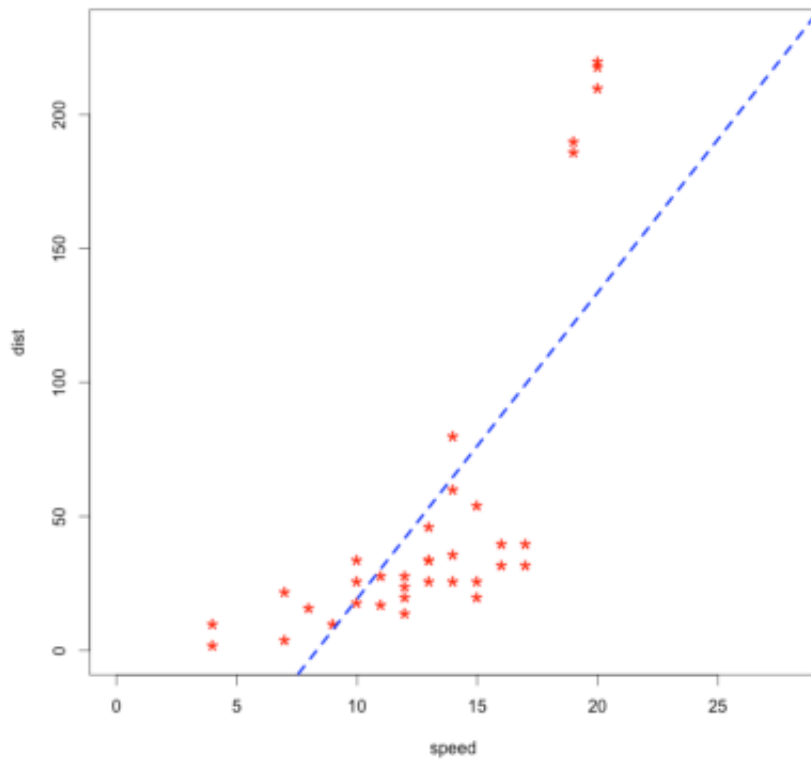
- **Datos:** un modelo neuronal es de tipo “caja negra”. Son modelos de interpolación (NUNCA DE EXTRAPOLACIÓN) que dependen FUERTEMENTE de calidad y cantidad de datos disponible.
- Calidad: relacionada con el grado con que los datos disponibles representan la función que se está aproximando. ***Deseable:*** obtenerlos siguiendo un plan de experiencias adecuadamente diseñado.



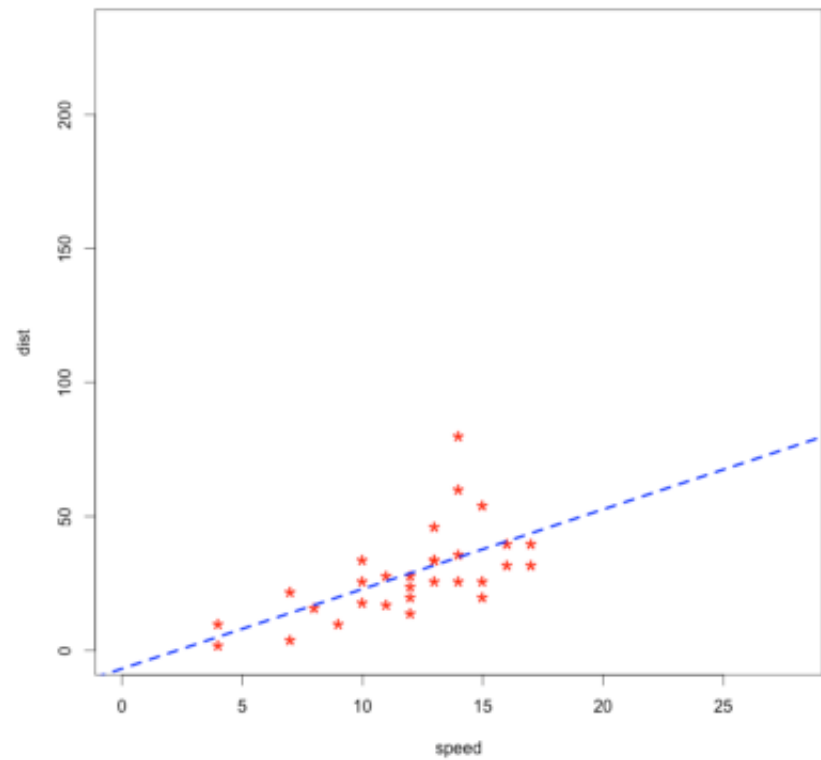
- **Datos (ejemplos):**
- Cantidad: es extremadamente importante pues sólo una cantidad de datos adecuada nos permitirá identificar en forma correcta los parámetros (pesos) de nuestro modelo neuronal.
- Si la cantidad de datos es pequeña, **NO PODEMOS PRETENDER** elaborar un modelo neuronal complejo.

- Examinar atentamente (visualmente) los datos. Detectar y en lo posible eliminar “outliers” (errores gruesos), vacíos, etc...
- Una atenta examinación visual permite a veces detectar correlación entre variables y, por lo tanto, reducir dimensionalidad, etc...

With Outliers

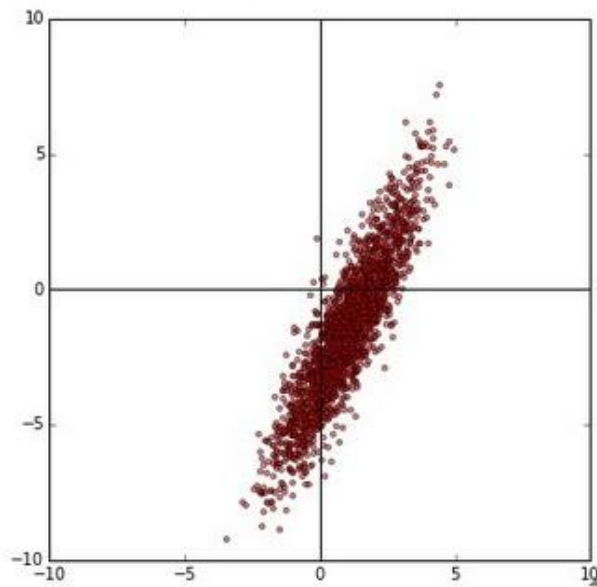


**Outliers removed
A much better fit!**

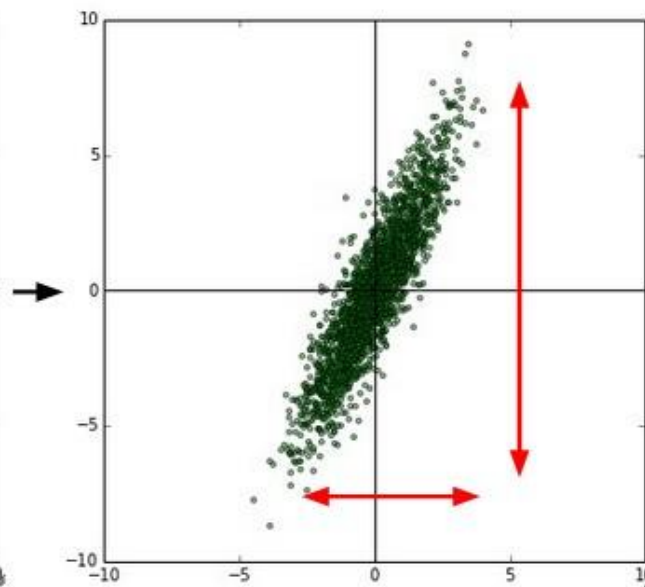


- **Normalización de variables:** Necesaria cuando intervienen variables con diferentes unidades y por lo tanto amplitudes a veces varios órdenes de magnitud diferentes.
- Ejemplo1 : $X_n = (X - X_{\min}) / (X_{\max} - X_{\min})$; $X_n \in (0,1)$
- Ejemplo2: $X_n = 2 * (X - X_{\min}) / (X_{\max} - X_{\min}) - 1$; $X_n \in (-1,1)$
- $X_{\max} = 1,1$ máximo valor de datos; $X_{\min} = 0,9$ mínimo valor de datos
- Es necesario, a la salida, realizar la desnormalización correspondiente.

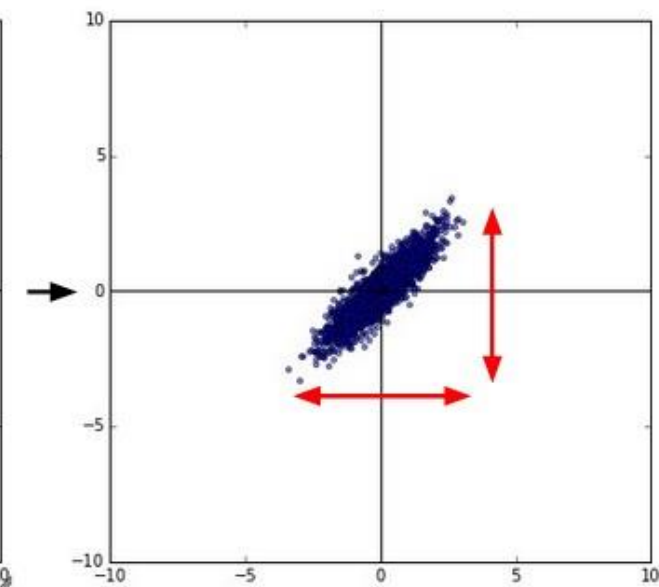
original data



zero-centered data



normalized data



Paso 3: Diseño del MLP

(problemas pequeños o medianos)

- Número de neuronas de entrada y de salida, dependiente del análisis de sistema anteriormente analizado.
- Número de neuronas de capa intermedia
Nc:
 - Lo importante es que dicho número de lugar a una cantidad de parámetros (pesos) N_w tal que:
 - $N_w < (\text{Número de ejemplos}) / 10$

- El número de pesos N_w de un perceptron multicapas, con 1 capa entrada con N_e neuronas, una capa oculta con N_c neuronas y una capa de salida con N_s neuronas es:

$$N_w = (N_e + 1) * N_c + (N_c + 1) * N_s$$

- Por lo tanto, si se tiene una red con 3 entradas, 4 neuronas en la capa oculta y 2 salidas, su cantidad de pesos es:

$$N_w = (3+1)*4+(4+1)*2 = 26$$

Luego, se requiere de **AL MENOS 260** ejemplos en el conjunto de entrenamiento para identificar los parámetros de esta red.

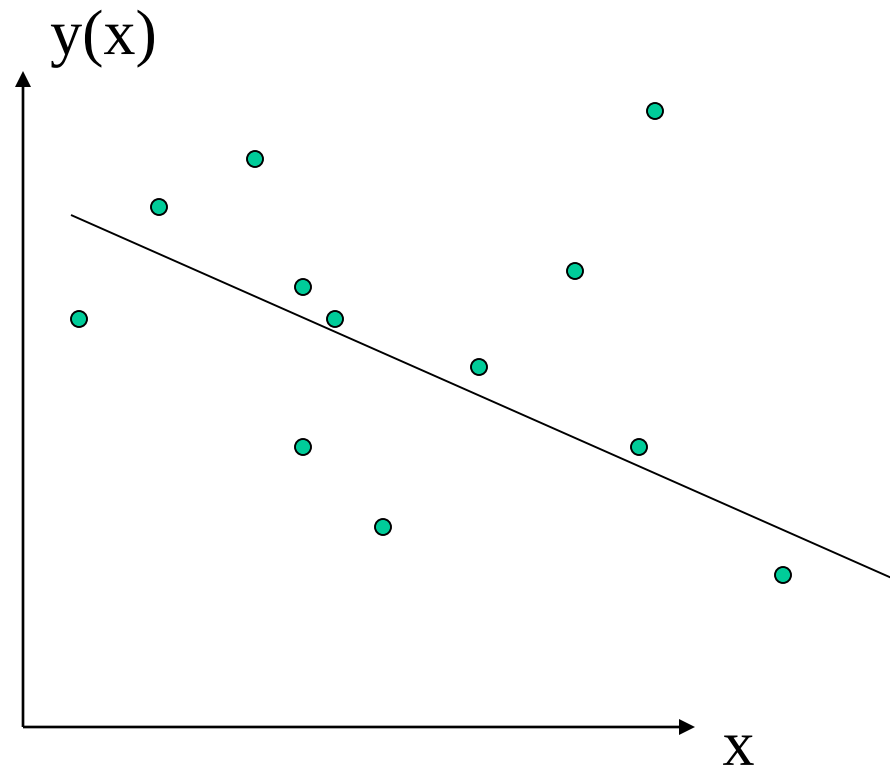
- **Funciones de transferencia:** la ventaja de los modelos neuronales y lo que los hace poderosos para aproximar funciones complejas es su capacidad de utilizar funciones de transferencia de tipo sigmoideal (facilita la retropropagación).
- Un criterio usual es que, dado que está demostrado que basta una sola capa oculta con un adecuado número de neuronas para aproximar con un grado de precisión arbitrario cualquier función no lineal [Funahashi, 89, Cybenko, 89, Hornik et al., 89, Hornik, 91], utilizar funciones sigmoideales en la capa oculta y funciones lineales en la de salida.
- Sin embargo también pueden usarse sigmoides en la salida.

Paso 4: Entrenamiento

- El entrenamiento supervisado de una red neuronal es un proceso muy delicado debido a la complejidad que ostenta la superficie de la función error, la que puede poseer numerosos mínimos locales, puntos silla, etc...

- Hay tres problemas principales que pueden surgir durante entrenamiento:
 - 1 Sesgo
 - 2.- Sobreparametrización
 - 3.- Sobreaprendizaje

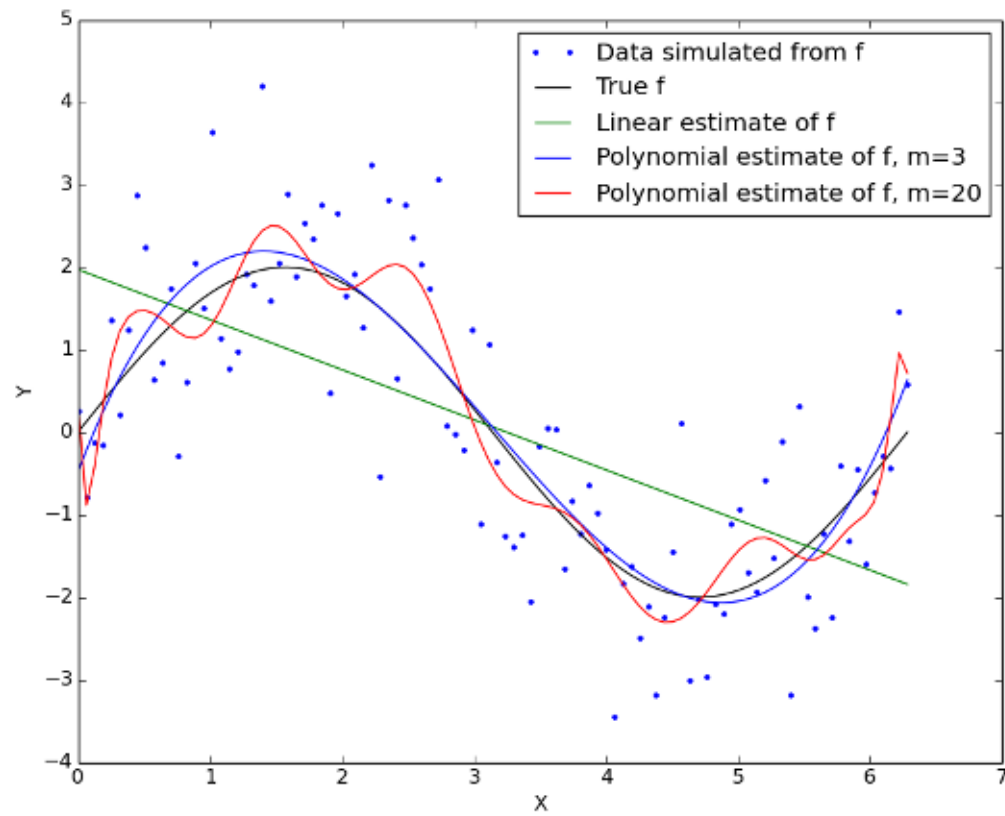
Los dos últimos dan lugar a un fenómeno similar que afecta la capacidad de “generalización” de la red (alta varianza).

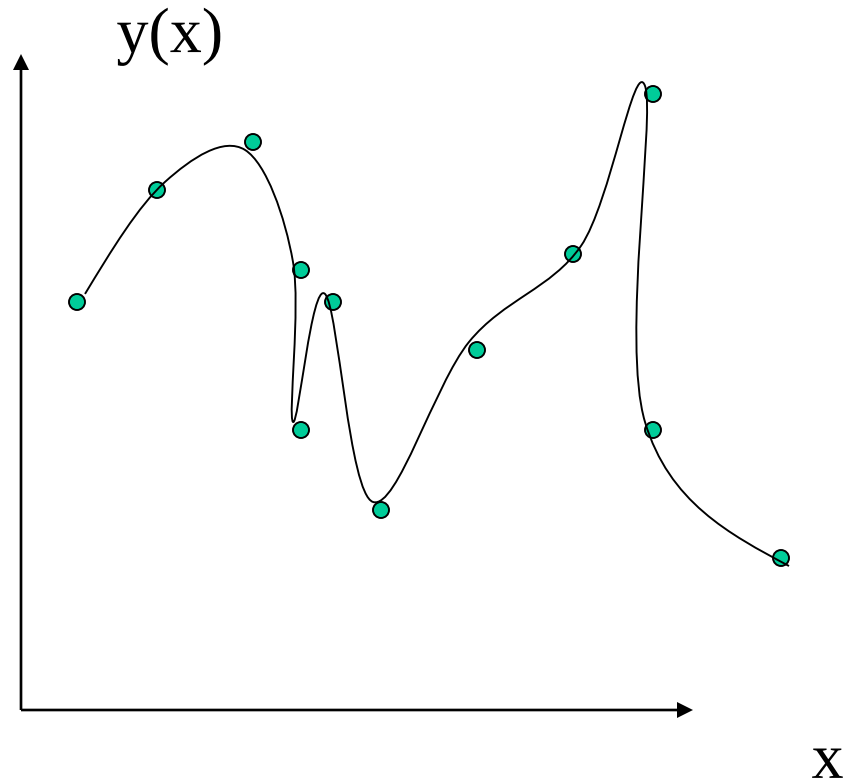


1.- Problema de gran sesgo (bias)

Como disminuir el sesgo?

- 1.- Logrando llegar a un mejor mínimo local para lo cual es importante realizar una buena cantidad de entrenamientos diferentes partiendo desde pesos iniciales aleatoriamente escogidos (20 o más intentos).
- 2.- Aumentando prudentemente el número de neuronas en la capa oculta





2.- Problema de gran varianza (sobreparametrización y sobreentrenamiento)

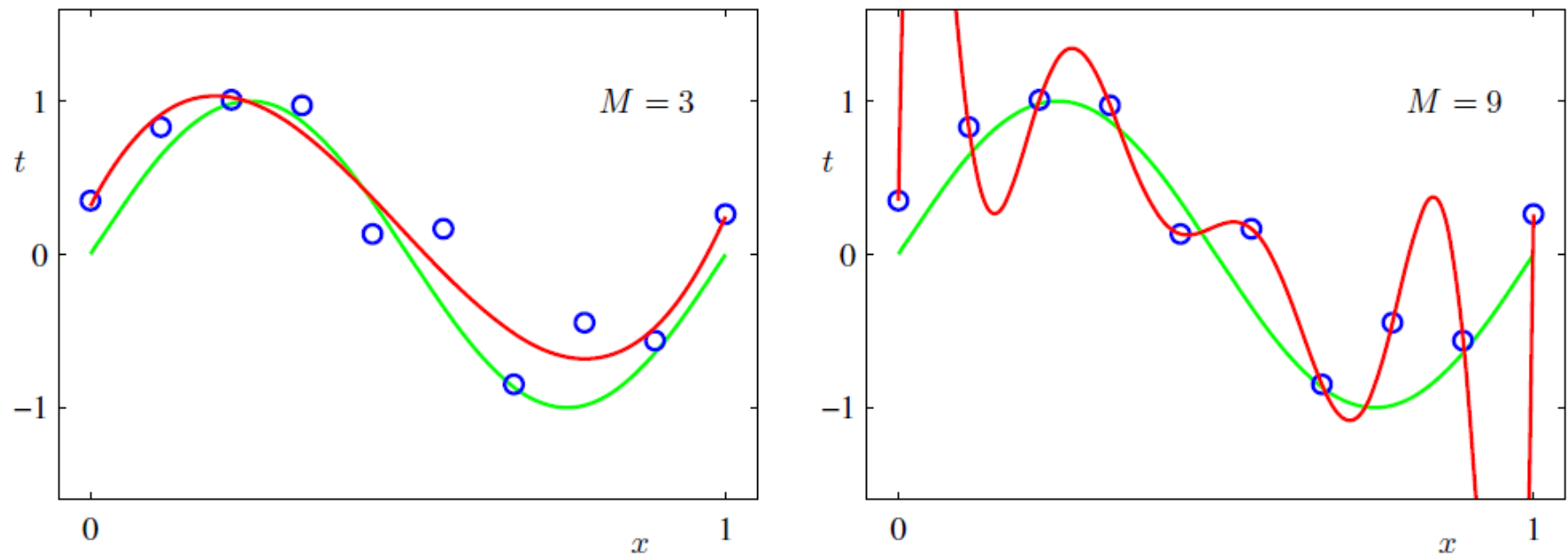
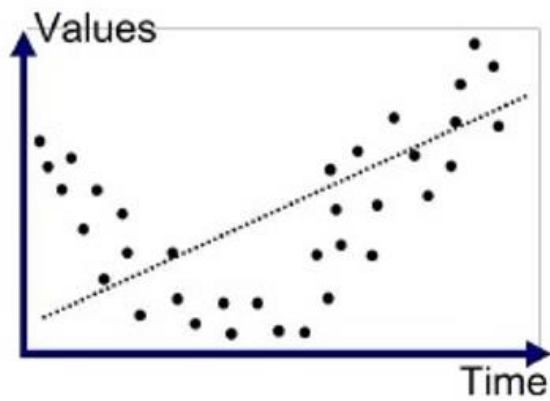
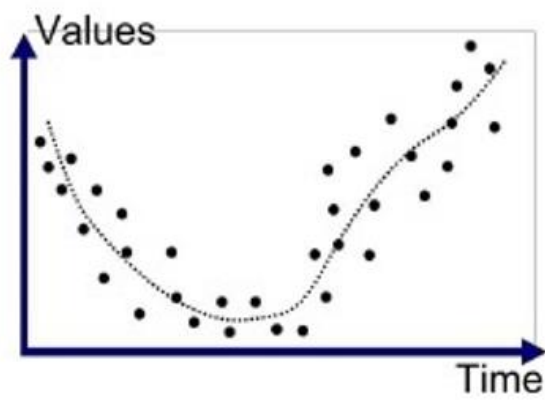


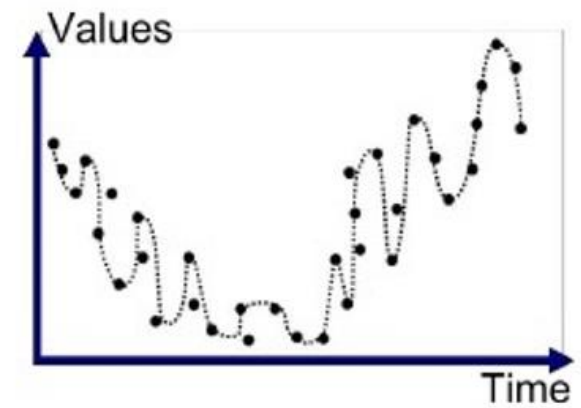
Figure 1.4 Plots of polynomials having various orders M , shown as red curves, fitted to the data set shown in Figure 1.2.



Underfitted



Good Fit/Robust



Overfitted

Formas de evitar los problemas anteriores:

1.- Trabajar siempre con dos conjuntos durante entrenamiento:

conjunto de entrenamiento

conjunto de test (prueba)

Lo ideal es visualizar el comportamiento de la función error en forma simultánea en ambos conjuntos

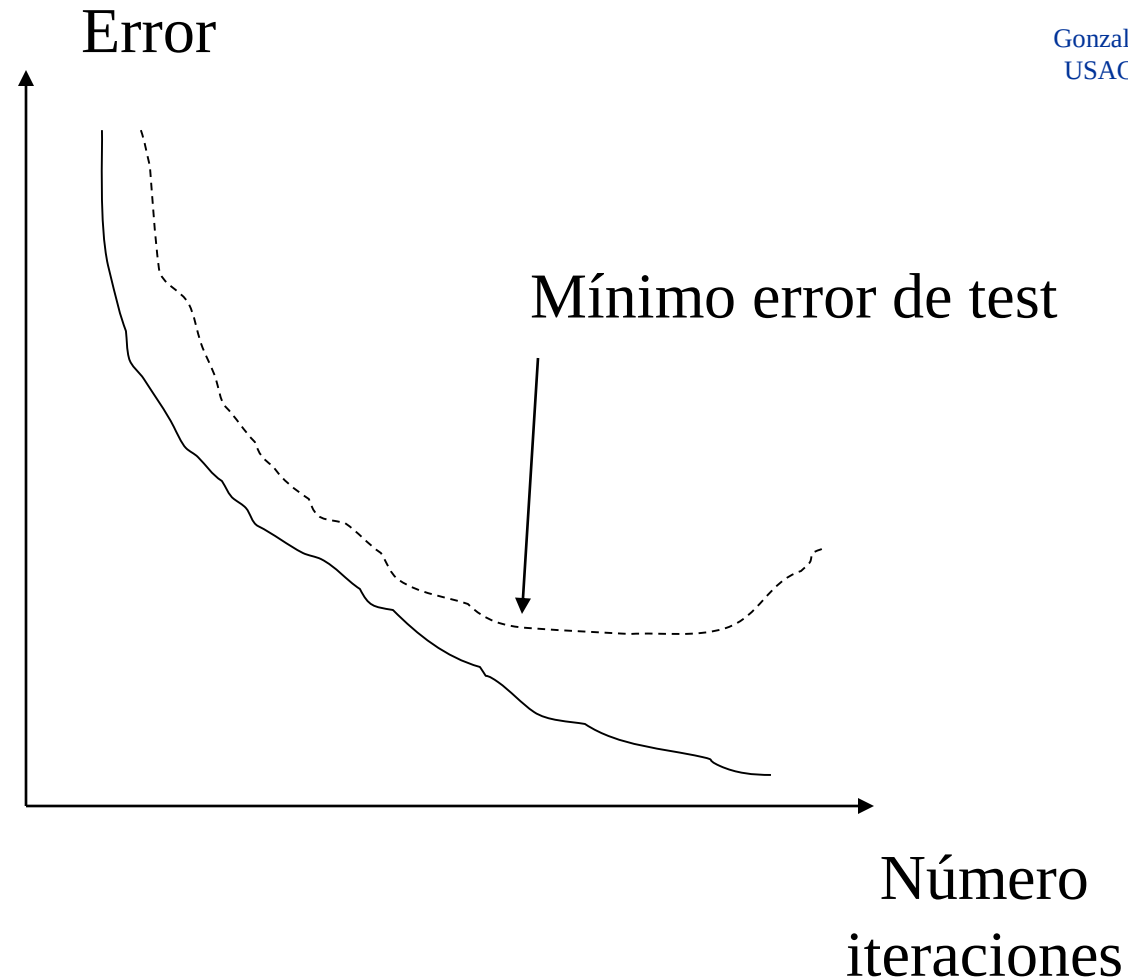
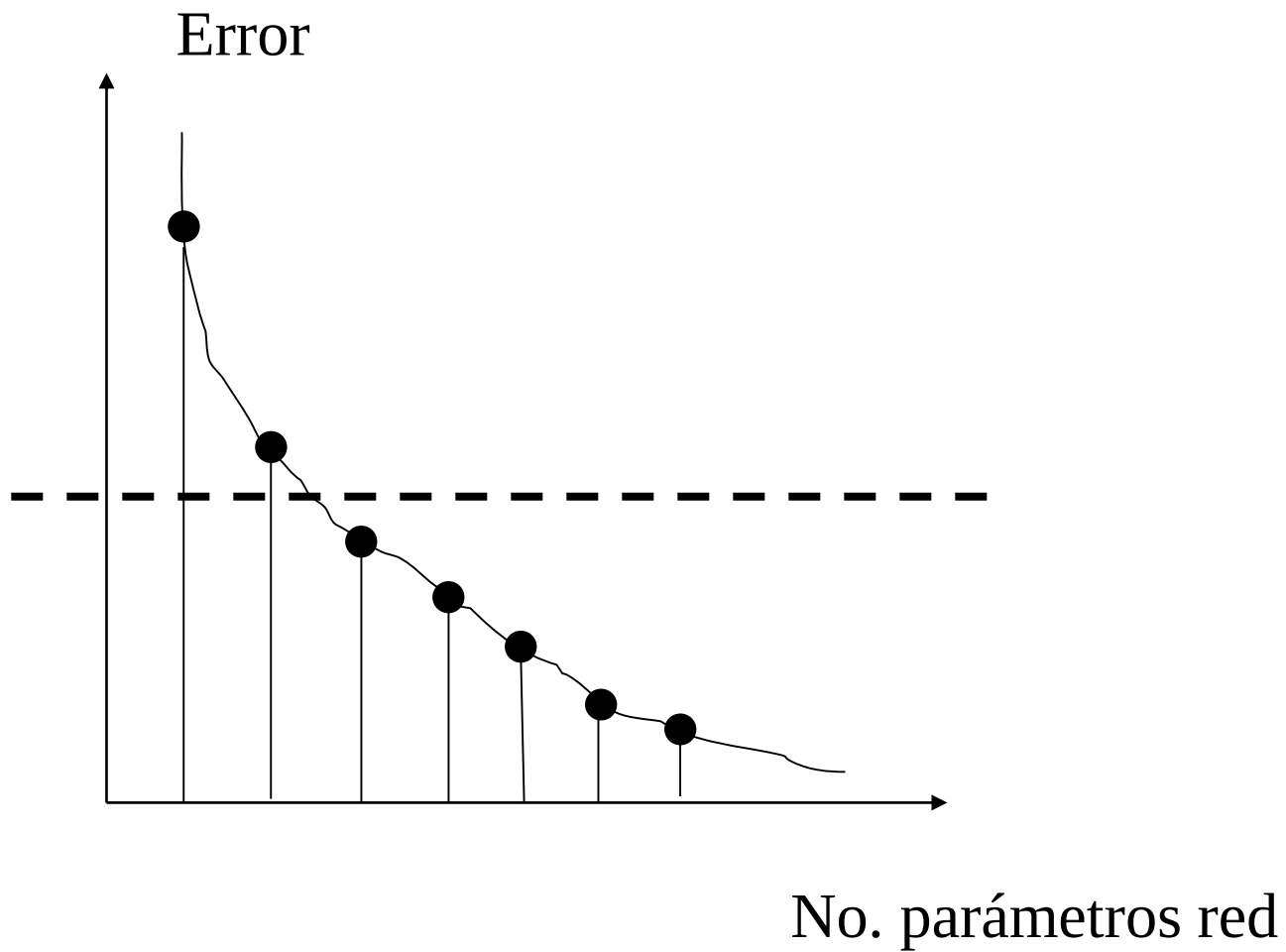


Grafico de función error en conjuntos
aprendizaje (-) y test (---)

1.- “**Early stopping**”: la idea es detener el proceso de aprendizaje apenas el error sobre el conjunto de test comience a aumentar. Se evita, de esta manera, que la red neuronal modelice el ruido que pueden contener los datos.

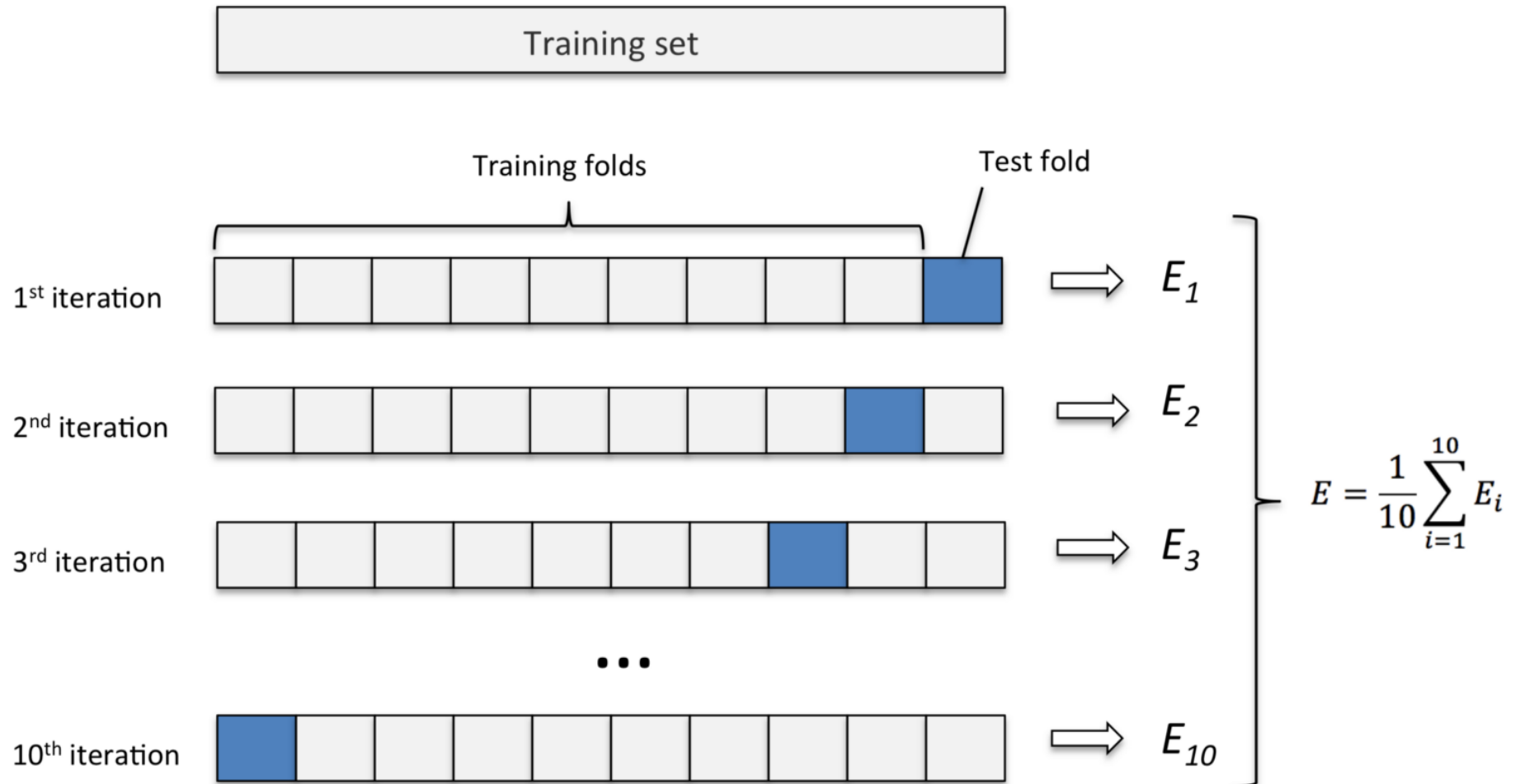


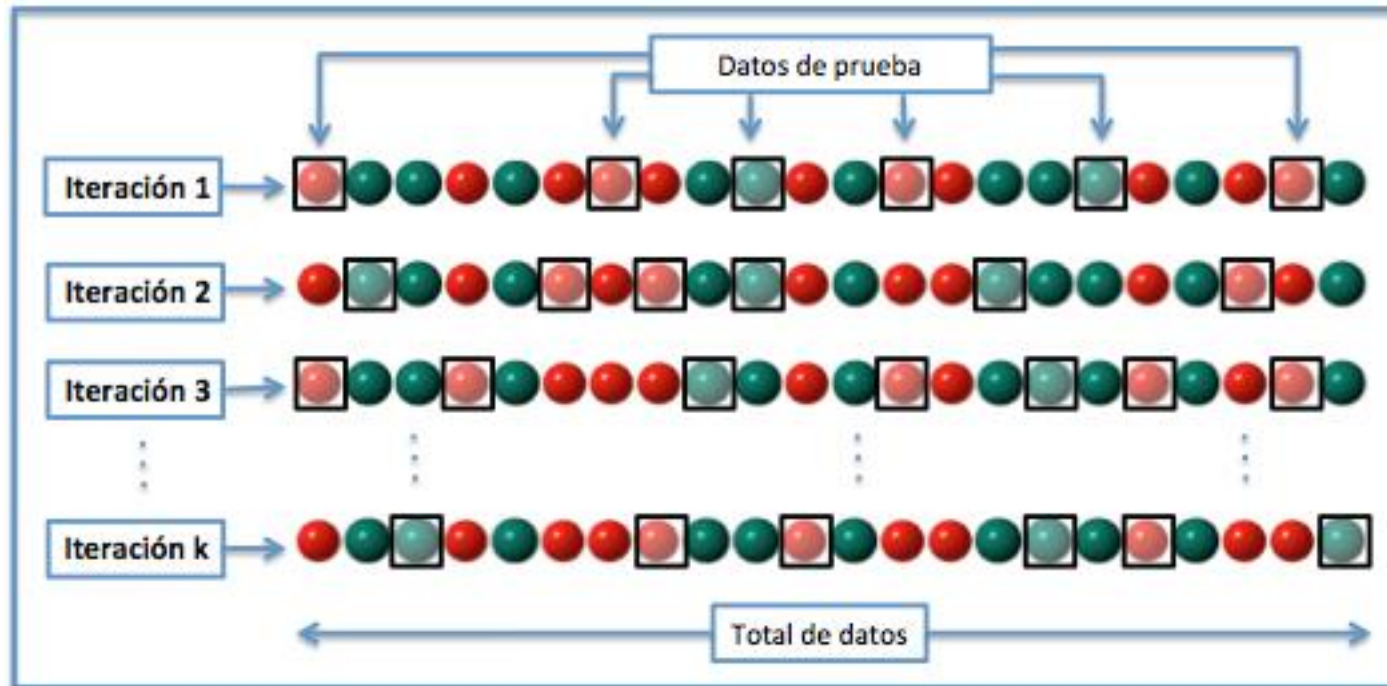
Característica de los conjuntos de aprendizaje y prueba:

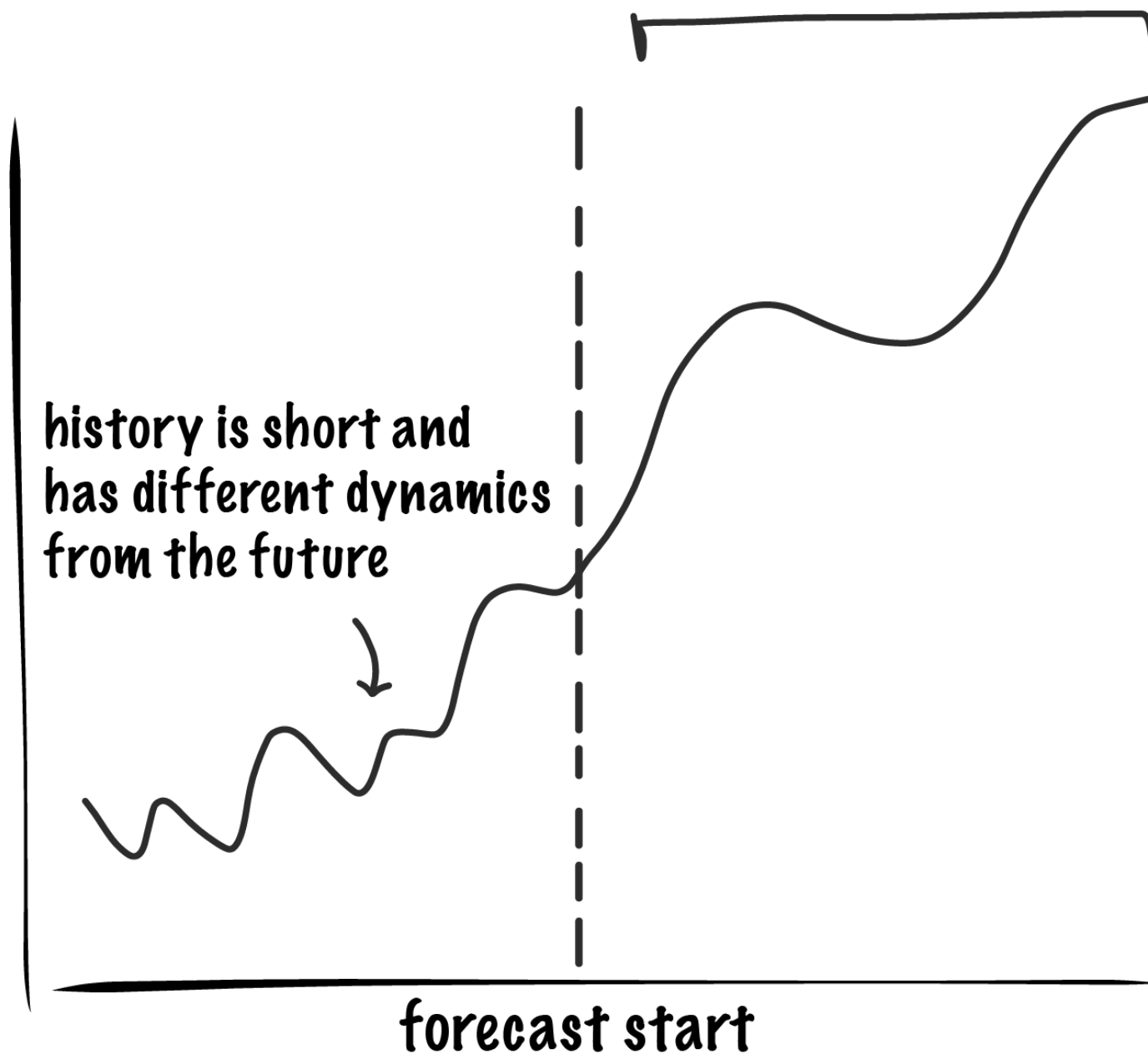
Ambos deben ser numerosos y los ejemplos que los compongan deben ser representativos del fenómeno subyacente que se quiere modelar

Otros métodos utilizados:

2.- Validación cruzada: se elaboran distintos modelos neuronales a partir del uso de los ejemplos disponibles para aprendizaje confeccionando de distinta forma los conjuntos de aprendizaje y test. Se escoge aquel modelo que da mínimo error sobre conjunto de test.







3.- **Poda:** la idea es comenzar con una red neuronal con una gran cantidad de pesos e ir “podando” dichos pesos bajo ciertos criterios que también incluyen la adición de términos a la función objetivo, los análisis de sensibilidad, etc...

4.- Regularización: se trata de agregar términos a la función objetivo de tal manera que al minimizarla se penalice la complejidad del modelo.

Regularización

Gonzalo Acuña L
USACH-DIINF

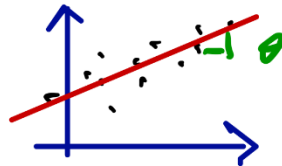
Mínimos cuadrados o MV
nos dan soluciones así:

$$\hat{\theta} = \underbrace{(X^T X)^{-1}}_{\substack{\text{dipende de inventar} \\ m \times d}} X^T y$$

$$\Rightarrow \hat{\theta} = (X^T X + \delta^2 I_d)^{-1} X^T y$$

$\Rightarrow$ Ridge Regression

$$J(\theta) = \underbrace{(y - X\theta)^T (y - X\theta)}_{\text{intenta minimizar } \theta \text{ (pendiente)}}$$



intenta
minimizar
 θ (pendiente)

Demo:

$$J(\theta) = (y - X\theta)^T (y - X\theta) + \delta^2 \theta^T \theta$$

$$\frac{\partial}{\partial \theta} J(\theta) = \frac{\partial}{\partial \theta} (\theta^T X^T X \theta - 2 y^T X \theta + y^T y + \delta^2 \theta^T \theta)$$

$$= 2X^T X \theta - 2X^T y + 2\delta^2 I \theta$$

$$= 2(X^T X + \delta^2 I) \theta - 2X^T y$$

Si igualamos a cero:

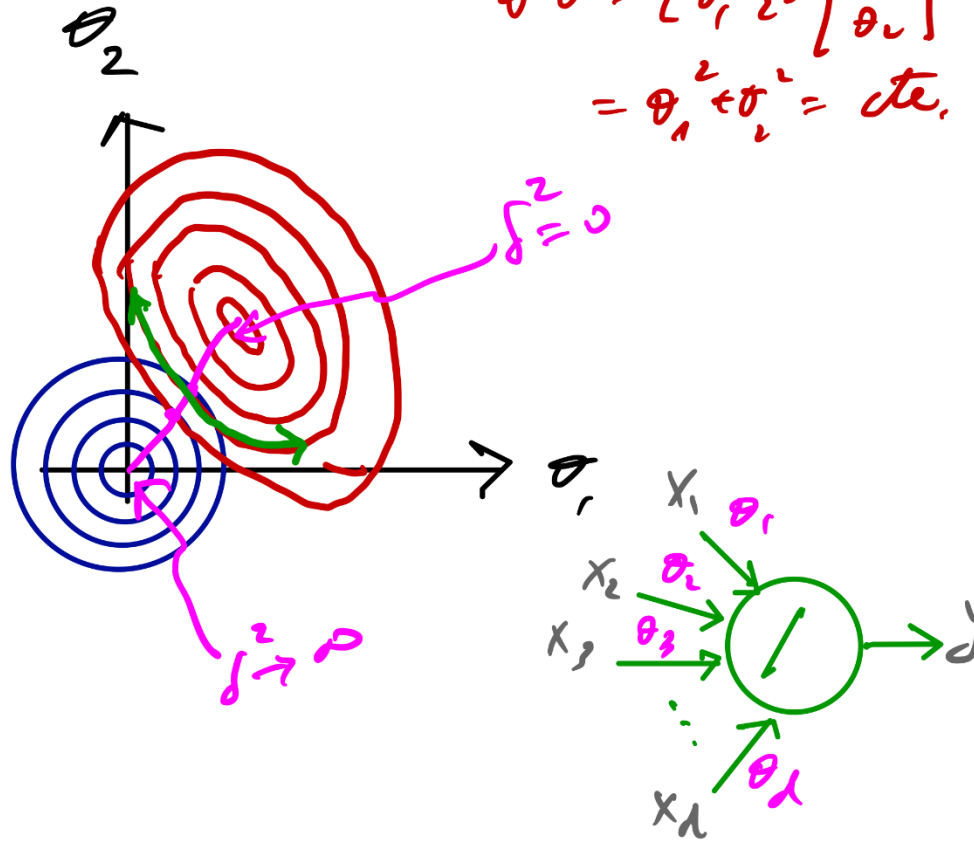
$$\hat{\theta}_{\text{ridge}} = (X^T X + \delta^2 I)^{-1} X^T y$$

Ridge Regression como Optimización con restricciones

Gonzalo Acuña L
USACH-DIINF

$$J(\theta) = \overbrace{(y - X\theta)^T (y - X\theta)} + \overbrace{\lambda^2 \theta^T \theta}$$
$$\equiv \min_{\theta^T \theta \leq t(\lambda)} \{ (y - X\theta)^T (y - X\theta) \}$$

$$\theta^T = [\theta_1 \quad \theta_2]$$
$$\theta^T \theta = [\theta_1 \quad \theta_2] \begin{bmatrix} \theta_1 \\ \theta_2 \end{bmatrix}$$
$$= \theta_1^2 + \theta_2^2 = r^2$$

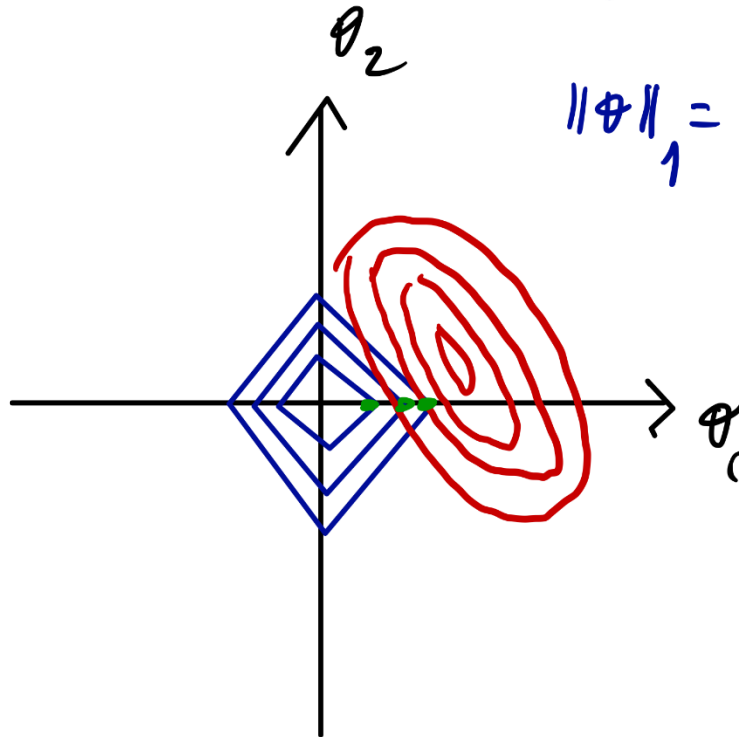


Regularización LASSO

Gonzalo Acuña L
USACH-DIINF

$$\theta^T \theta = \|\theta\|_2^2$$

$$\|\theta\|_1 = \sum |\theta_i|$$

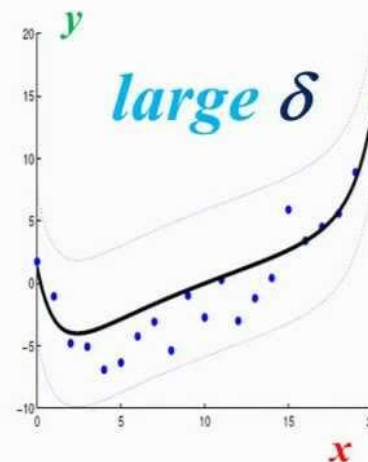
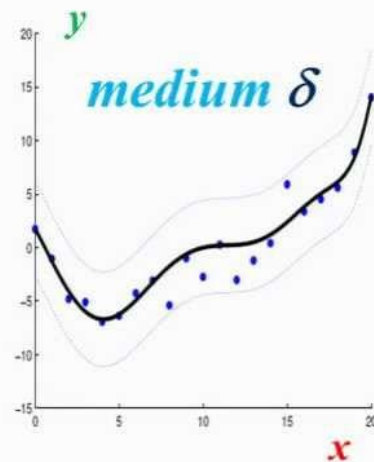
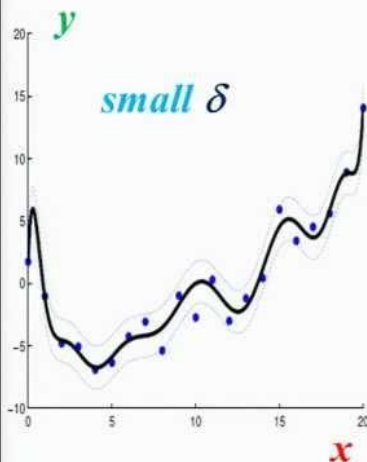


Example: Ridge regression with a polynomial of degree 14

$$\hat{y}(x_i) = 1 \theta_0 + x_i \theta_1 + x_i^2 \theta_2 + \dots + x_i^{13} \theta_{13} + x_i^{14} \theta_{14}$$

$$\Phi = [1 \ x_i \ x_i^2 \ \dots \ x_i^{13} \ x_i^{14}]$$

$$J(\theta) = (y - \Phi \theta)^T (y - \Phi \theta) + \delta^2 \theta^T \theta$$



Computer
Science

© Copyright

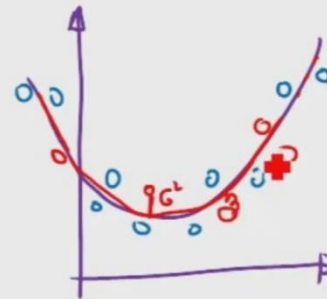
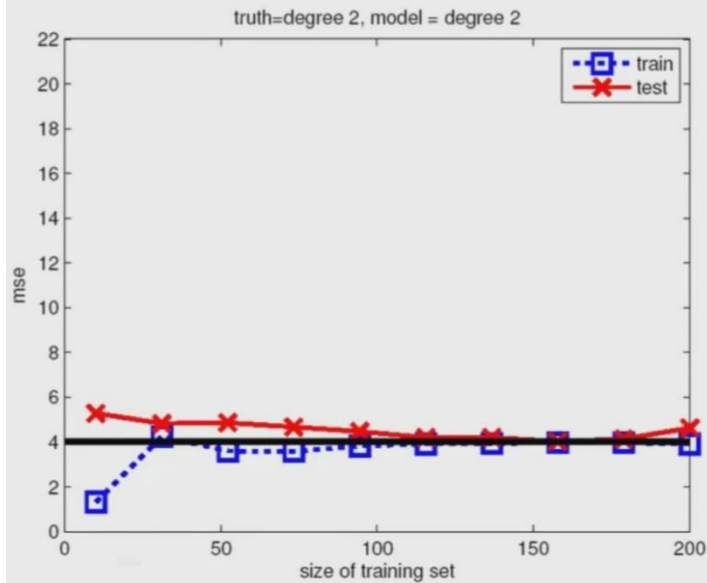
CPSC 540
Nando De Freitas

Jan 17 2013
13:27

Effect of data when we have the right model

$$y_i = \theta_0 + x_i \theta_1 + x_i^2 \theta_2 + \mathcal{N}(0, \sigma^2)$$

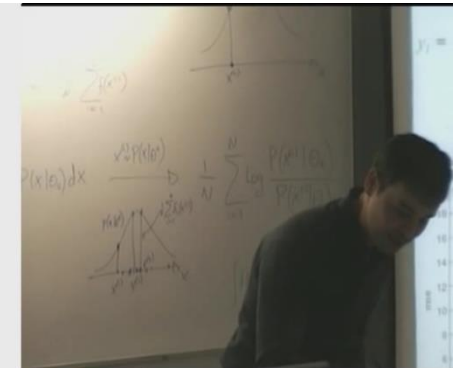
$$\hat{y} = \theta_0 + x_i \theta_1 + x_i^2 \theta_2$$



○ Train data

● Test data

noise
Error



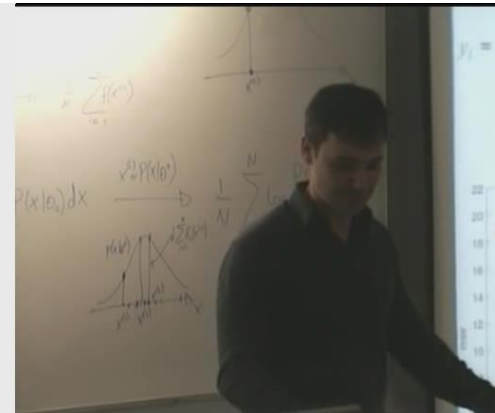
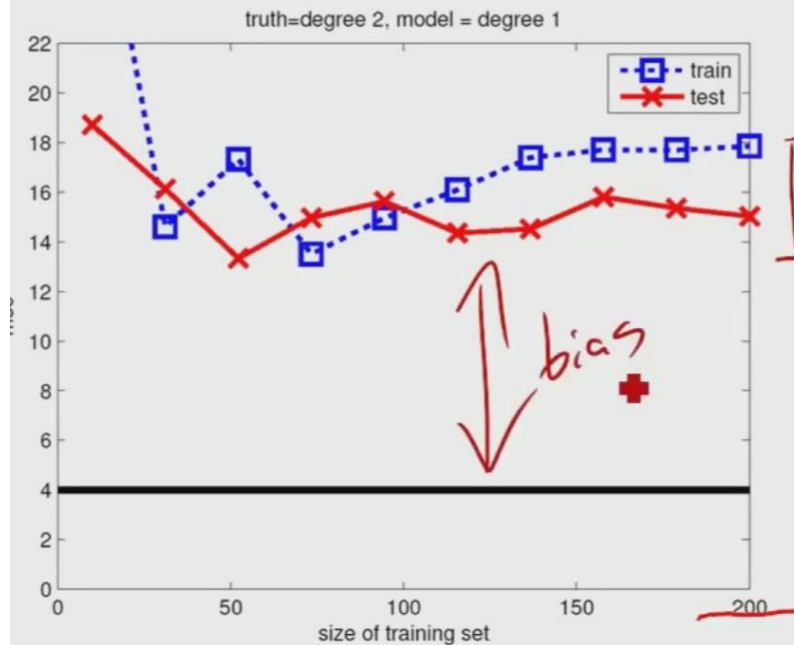
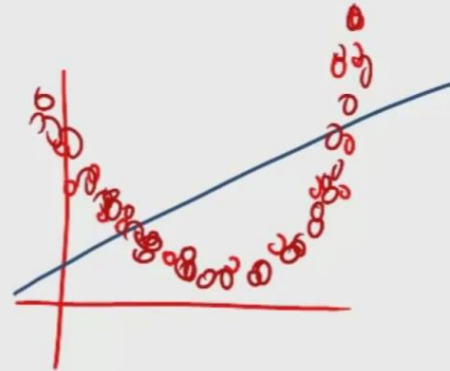
DEPARTMENT OF
**COMPUTER
SCIENCE**

Deep Learning
Nando de Freitas
29 January 2015

Effect of data when the model is too simple

$$y_i = \theta_0 + x_i \theta_1 + x_i^2 \theta_2 + \mathcal{N}(0, \sigma^2)$$

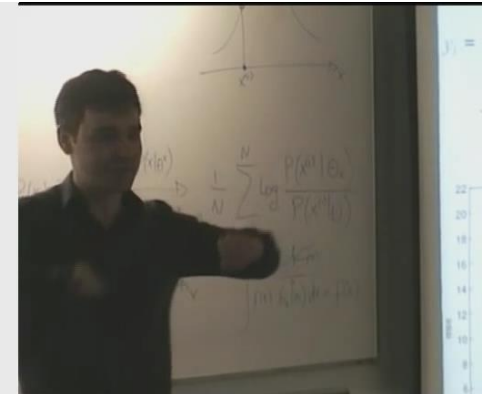
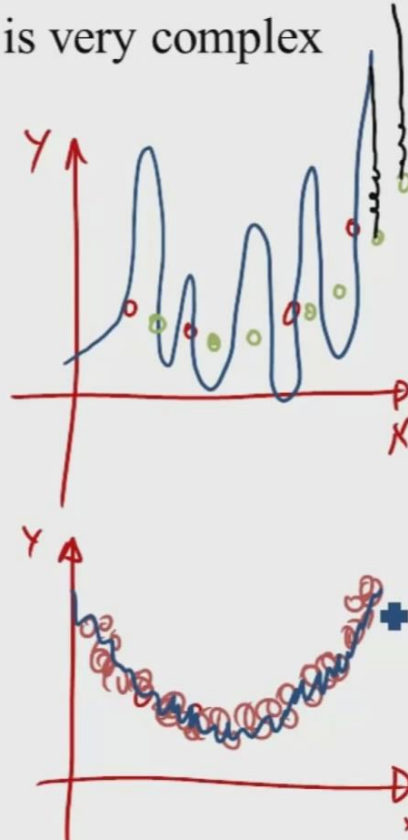
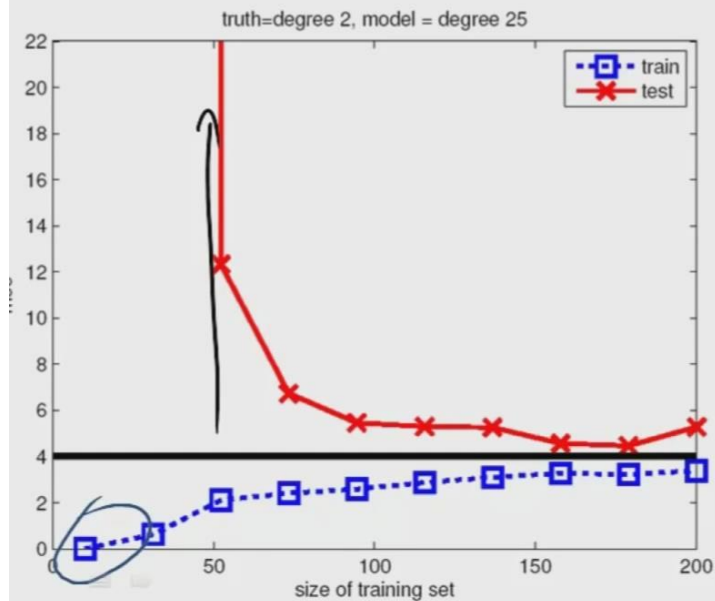
$$\hat{y} = \theta_0 + x_i \theta_1$$



Effect of data when the model is very complex

$$v_i = \theta_0 + x_i \theta_1 + x_i^2 \theta_2 + \mathcal{N}(0, \sigma^2)$$

$$\hat{y}_i = \theta_0 + x_i \theta_1 + \dots + x_i^{25} \theta_{25}$$



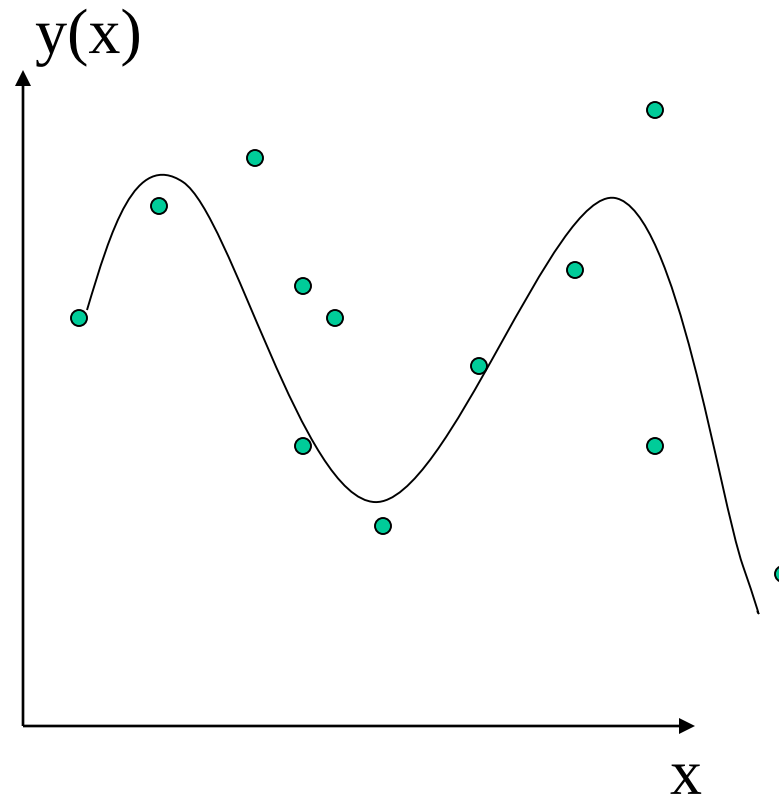
DEPARTMENT OF
**COMPUTER
SCIENCE**

Deep Learning
Nando de Freitas
29 January 2015

Paso 5: Generalización

Para probar la capacidad de generalización de la red, o sea sus resultados sobre un conjunto distinto de datos, es importante haber reservado ejemplos para confeccionar un tercer conjunto, el **CONJUNTO DE GENERALIZACIÓN**.

Este debe ser tan representativo del fenómeno a modelar como los anteriores (aprendizaje y test).



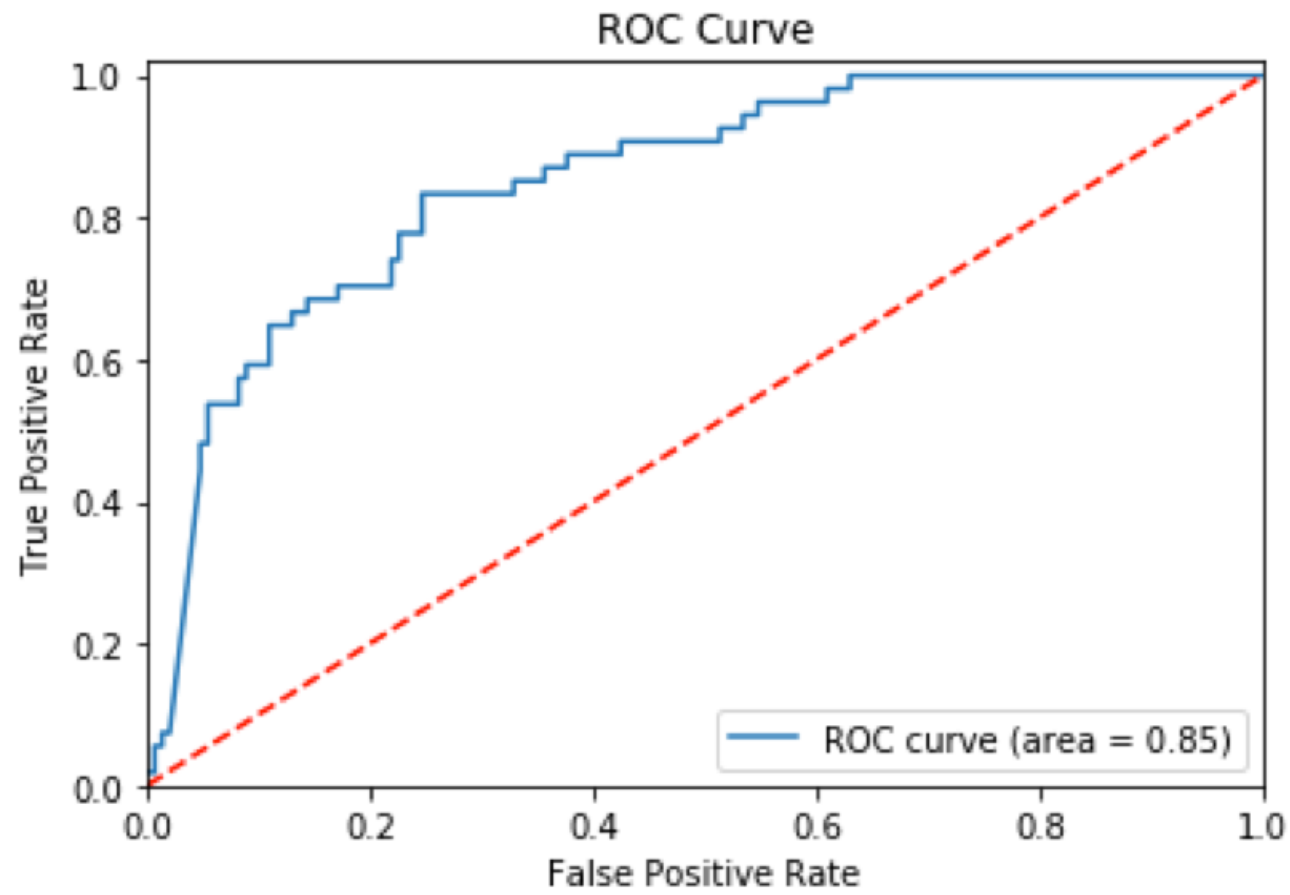
3.- Aproximación correcta de la función subyacente →
GENERALIZACIÓN

Es importante chequear la capacidad de generalización, lo que también se conoce como **VALIDACIÓN** del modelo neuronal.

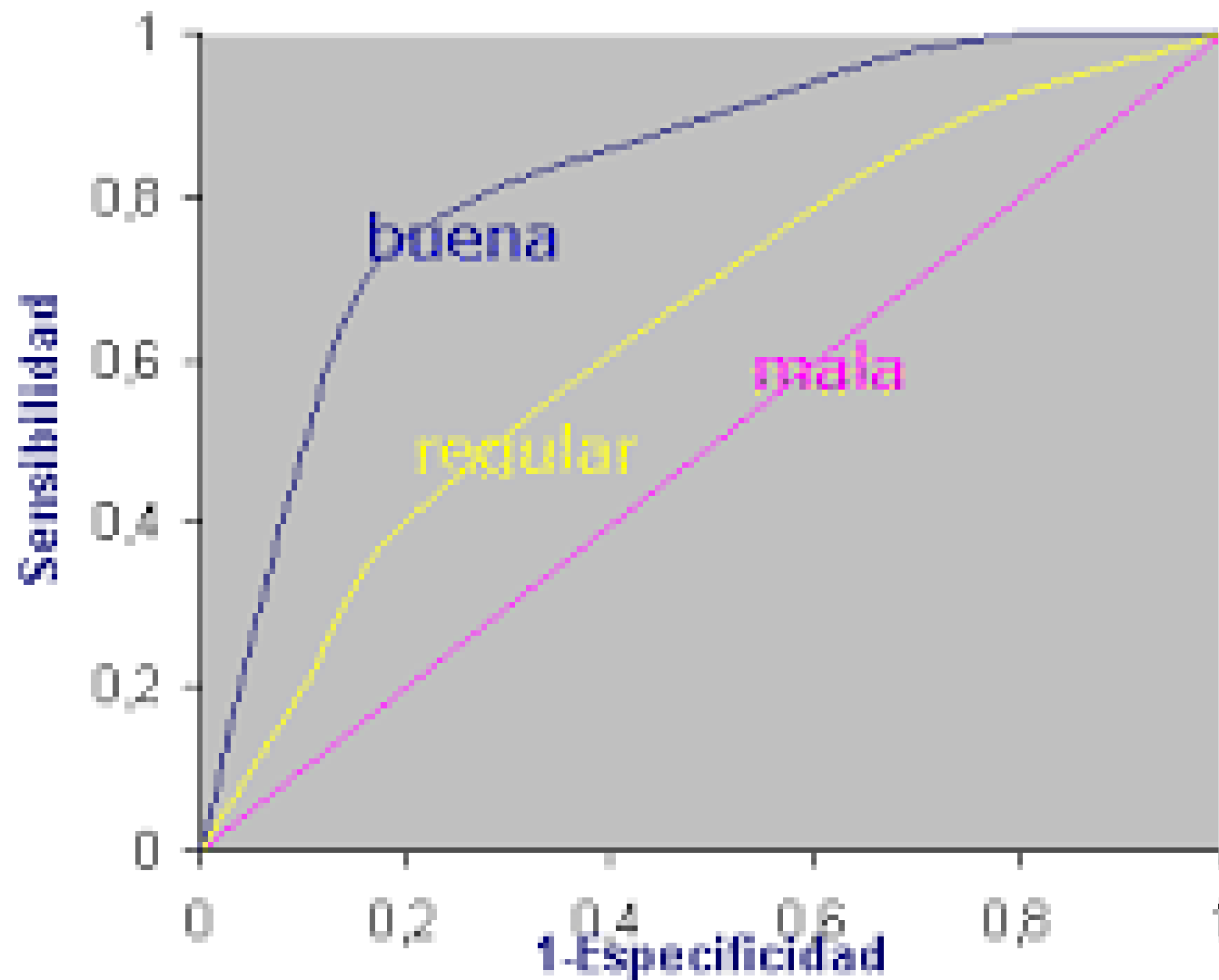
Índices Validación Clasificadores

Matriz Confusión

	Actual - Cancer	Actual - NOT Cancer	Total
Predicted - Cancer	TP = 20	FP = 70	90
Predicted - NOT Cancer	FN = 10	TN = 200	210
Total	30	270	300



Tipos de curvas ROC



Índices para validación numérica de modelos (regresión)

Gonzalo Acuña L
USACH-DIINF

Índices de error usados:

$$IA = 1 - \frac{\sum_{i=1}^n (o_i - p_i)^2}{\sum_{i=1}^n (|o_i'| + |p_i'|)^2}$$

$$RMS = \sqrt{\frac{\sum_{i=1}^n (o_i - p_i)^2}{\sum_{i=1}^n o_i^2}}$$

$$RSD = \sqrt{\frac{\sum_{i=1}^n (o_i - p_i)^2}{N}}$$

O_i : Valores predichos.

p_i : Valores observados

N : Número total de datos.

O_m : Valor medio de las observaciones.

$$o_i' = o_i - O_m$$

$$p_i' = p_i - O_m$$